

Result

Size

Time

Cycles

GPU

SM Frequency

Process

Attributes

Current

643 - ik_batch_solve_mixed

(100, 1, 1)x(128, 1, 1)

827.01 us

1,860,686

0 - NVIDIA GeForce RTX 4060 Laptop GPU

2.25 Ghz

[4033621] standard_robot_cuda_runner_A7

GPU Compute Capability: 8.9

Summary

Details

Source

Context

Comments

Raw

Session

Compare

Tools

View

Export

GPU Speed Of Light Throughput

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

Compute (SM) Throughput [%]	60.73	Duration [us]	827.01
Memory Throughput [%]	1.42	Elapsed Cycles [cycle]	1,860,686
L1/TEX Cache Throughput [%]	1.92	SM Active Cycles [cycle]	1,372,031.54
L2 Cache Throughput [%]	0.53	SM Frequency [Ghz]	2.25
DRAM Throughput [%]	1.16	DRAM Frequency [Ghz]	7.99

High Compute Throughput

Compute is more heavily utilized than Memory. Look at the [► Compute Workload Analysis](#) section to see what the compute pipelines are spending their time doing. Also, consider whether any computation is redundant and could be reduced or moved to look-up tables.

Key Performance Indicators

Roofline Analysis

The ratio of peak float (FP32) to double (FP64) performance on this device is 64:1. The workload achieved close to 0% of this device's FP32 peak performance and close to 1% of its FP64 peak performance. See the [🔗 Profiling Guide](#) for more details on roofline analysis.

PM Sampling

Timeline view of PM metrics sampled periodically over the workload duration. Data is collected across multiple passes. Use this section to understand how workload behavior changes over its runtime.

Maximum Sampling Interval [us]	1	# Pass Groups	3
Maximum Buffer Size [Mbyte]	8.39	-	-

Compute Workload Analysis

Detailed analysis of the compute resources of the streaming multiprocessors (SM), including the achieved instructions per clock (IPC) and the utilization of each available pipeline. Pipelines with very high utilization might limit the overall performance.

Executed Ipc Elapsed [inst/cycle]	0.19	SM Busy [%]	60.73
Executed Ipc Active [inst/cycle]	0.26	Issue Slots Busy [%]	4.77
Issued Ipc Active [inst/cycle]	0.26		

High Utilization

FP64 is the highest-utilized pipeline (60.7%) based on elapsed cycles in the workload, taking into account the rates of its different instructions. It executes 64-bit floating point operations. The pipeline is well-utilized, but might become a bottleneck if more work is added. Based on the number of executed instructions, the highest utilized pipeline (60.7%) is FP64 (FP64). It executes non-DMMA 64-bit floating point operations. Comparing the two, the overall pipeline utilization appears to be caused by frequent, low-latency instructions. See the [🔗 Profiling Guide](#) or hover over the pipeline name to understand the workloads handled by each pipeline. The [► Instruction Statistics](#) section shows the mix of executed instructions for this workload. Check the [► Warp State Statistics](#) section for which reasons cause warps to stall.

Key Performance Indicators

Memory Workload Analysis

Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.

Memory Throughput [Gbyte/s]	2.98	Mem Busy [%]	0.94
L1/TEX Hit Rate [%]	98.62	Max Bandwidth [%]	1.42
L2 Hit Rate [%]	33.87	Mem Pipes Busy [%]	14.63
L2 Compression Input Sectors [sector]	0	Local Memory Spilling Requests	0
L2 Compression Ratio	0	Shared Memory Spilling Requests	0
L2 Compression Success Rate [%]	0	Local Memory Spilling Request Overhead [%]	0
L2 Persisting Size [Mbyte]	6.29	Shared Memory Spilling Request Overhead [%]	0
L2 Sector Promotion Misses [%]	0	-	-

Local Memory Usage

Est. Speedup: 20.38%

This workload accesses [🔗 local memory](#), accounting for 72.26% of all sectors requested in L1TEX. Local memory refers to a memory space that is private to each thread. Although it is logically distinct, local memory resides in global memory. This can lead to higher memory latency compared to the usage of registers or shared memory, a decrease in available bandwidth to global memory, and an increase in executed instructions. The local memory accesses of this workload are largely cached in L1TEX (99.94% of loads, and 96.07% of stores). However, even spilling to L1TEX might lead to high performance penalties compared with staying in registers. One common reason for the use of local memory is arrays being too large to fit into registers or not being indexed using compile time constants (see the "Local Memory" section of the [🔗 CUDA Programming Guide](#). For more advice on how to reduce the usage of local memory refer to the [🔗 documentation](#).

Key Performance Indicators

L1TEX Global Load Access Pattern

Est. Speedup: 0.84%

The memory access pattern for global loads from L1TEX might not be optimal. On average, only 13.0 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the [► Source Counters](#) section for uncoalesced global loads.

Key Performance Indicators

L1TEX Global Store Access Pattern

Est. Speedup: 0.78%

The memory access pattern for global stores to L1TEX might not be optimal. On average, only 14.4 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the [► Source Counters](#) section for uncoalesced global stores.

Key Performance Indicators

L1TEX Local Load Access Pattern

Est. Speedup: 1.24%

The memory access pattern for local loads from L1TEX might not be optimal. On average, only 4.0 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the [► Source Counters](#) section for uncoalesced local loads.

Key Performance Indicators

L1TEX Local Store Access Pattern

Est. Speedup: 1.24%

The memory access pattern for local stores to L1TEX might not be optimal. On average, only 4.0 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the [► Source Counters](#) section for uncoalesced local stores.

Key Performance Indicators

Scheduler Statistics

Summary of the activity of the schedulers issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions for. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks the state of the allocated warps in the pool (Active Warps). Active warps that are not stalled (Eligible Warps) are ready to issue their next instruction. From the set of eligible warps the scheduler selects a single warp from which to issue one or more instructions (Issued Warp). On cycles with no eligible warps, the issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates poor latency hiding.

Active Warps Per Scheduler [warp]	3.49	No Eligible [%]	93.53
Eligible Warps Per Scheduler [warp]	0.07	One or More Eligible [%]	6.47
Issued Warp Per Scheduler	0.06		

Issue Slot Utilization

Est. Local Speedup: 39.27%

Every scheduler is capable of issuing one instruction per cycle, but for this workload each scheduler only issues an instruction every 15.5 cycles. This might leave hardware resources underutilized and may lead to less optimal performance. Out of the maximum of 12 warps per scheduler, this workload allocates an average of 3.49 active warps per scheduler, but only an average of 0.07 warps were eligible per cycle. Eligible warps are the subset of active warps that are ready to issue their next instruction. Every cycle with no eligible warp results in no instruction being issued and the issue slot remains unused. To increase the number of eligible warps, avoid possible load imbalances due to highly different execution durations per warp. Reducing stalls indicated on the [► Warp State Statistics](#) and [► Source Counters](#) sections can help, too.

Key Performance Indicators

Warp State Statistics

Analysis of the states in which all warps spent cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction define the latency between two consecutive instructions. The higher the value, the more warp parallelism is required to hide this latency. For each warp state, the chart shows the average number of cycles spent in that state per issued instruction. Stalls are not always impacting the overall performance nor are they completely avoidable. Only focus on stall reasons if the schedulers fail to issue every cycle. When executing a kernel with mixed library and user code, these metrics show the combined values.

Warp Cycles Per Issued Instruction [cycle]	53.95	Avg. Active Threads Per Warp	5.15
Warp Cycles Per Executed Instruction [cycle]	54.03	Avg. Not Predicated Off Threads Per Warp	4.61

Barrier Stalls

Est. Speedup: 39.27%

On average, each warp of this workload spends 39.7 cycles being stalled waiting for sibling warps at a CTA barrier. A high number of warps waiting at a barrier is commonly caused by diverging code paths before a barrier. This causes some warps to wait a long time until other warps reach the synchronization point. Whenever possible, try to divide up the work into blocks of uniform workloads. If the block size is 512 threads or greater, consider splitting it into smaller groups. This can increase eligible warps without affecting occupancy, unless shared memory becomes a new occupancy limiter. Also, try to identify which barrier instruction causes the most stalls, and optimize the code executed before that synchronization point first. This stall type represents about 73.6% of the total average of 54.0 cycles between issuing two instructions.

Key Performance Indicators

Warp Stall

Check the [► Warp Stall Sampling \(All Samples\)](#) table for the top stall locations in your source based on sampling data. The [🔗 Profiling Guide](#) provides more details on each stall reason.

Thread Divergence

Est. Speedup: 51.98%

Instructions are executed in warps, which are groups of 32 threads. Optimal instruction throughput is achieved if all 32 threads of a warp execute the same instruction. The chosen launch configuration, early thread completion, and divergent flow control can significantly lower the number of active threads in a warp per cycle. This workload achieves an average of 5.2 threads being active per cycle. This is further reduced to 4.6 threads per warp due to predication. The compiler may use predication to avoid an actual branch. Instead, all instructions are scheduled, but a per-thread condition code or predicate controls which threads execute the instructions. Try to avoid different execution paths within a warp when possible.

Key Performance Indicators

Instruction Statistics

Statistics of the executed low-level assembly instructions (SASS). The instruction mix provides insight into the types and frequency of the executed instructions. A narrow mix of instruction types implies a dependency on few instruction pipelines, while others remain unused. Using multiple pipelines allows hiding latencies and enables parallel execution. Note that 'Instructions/Opcode' and 'Executed Instructions' are measured differently and can diverge if cycles are spent in system calls.

Executed Instructions [inst]	8,511,833	Avg. Executed Instructions Per Scheduler [inst]	88,664.93
Issued Instructions [inst]	8,523,800	Avg. Issued Instructions Per Scheduler [inst]	88,789.58
Local Memory Spilling Requests [inst]	0	Shared Memory Spilling Requests [inst]	0

FP32 Non-Fused Instructions

Est. Speedup: 0.33%

This workload executes 2741862 fused and 1134545 non-fused FP32 instructions. By converting pairs of non-fused instructions to their [🔗 fused](#), higher-throughput equivalent, the achieved FP32 performance could be increased by up to 15% (relative to its current performance).

Key Performance Indicators

FP64 Non-Fused Instructions

Est. Speedup: 7.61%

This workload executes 738324 fused and 246799 non-fused FP64 instructions. By converting pairs of non-fused instructions to their [🔗 fused](#), higher-throughput equivalent, the achieved FP64 performance could be increased by up to 13% (relative to its current performance).

Key Performance Indicators

NVLink Topology

NVLink Topology diagram shows logical NVLink connections with transmit/receive throughput.

NVLink Tables

Detailed tables with properties for each NVLink.

NUMA Affinity

Non-uniform memory access (NUMA) affinities based on compute and memory distances for all GPUs.

Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	100	Function Cache Configuration	CachePreferNone
Registers Per Thread [register/thread]	98	Static Shared Memory Per Block [byte/block]	820
Block Size	128	Dynamic Shared Memory Per Block [byte/block]	0
Threads [thread]	12,800	Driver Shared Memory Per Block [kbyte/block]	1.02
Waves Per SM	1.04	Shared Memory Configuration Size [kbyte]	16.38
Uses Green Context	0	Stack Size	1,024
# SMs [SM]	24	# TPCs	12
Enabled TPC IDs	all	-	-

Tail Effect

Est. Speedup: 50.00%

A wave of thread blocks is defined as the maximum number of blocks that can be executed in parallel on the target GPU. The number of blocks in a wave depends on the number of multiprocessors and the theoretical occupancy of the kernel. This kernel launch results in 1 full waves and a partial wave of 5 thread blocks. Under the assumption of a uniform execution duration of all thread blocks, this partial wave may account for up to 50.0% of the total runtime of this kernel. Try launching a grid with no partial wave. The overall impact of this tail effect also lessens with the number of full waves executed for a grid. See the [🔗 Hardware Model](#) description for more details on launch configurations.

Key Performance Indicators

Occupancy

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	33.33	Block Limit Registers [block]	4
Theoretical Active Warps per SM [warp]	16	Block Limit Shared Mem [block]	8
Achieved Occupancy [%]	29.06	Block Limit Warps [block]	12
Achieved Active Warps Per SM [warp]	13.95	Block Limit SM [block]	24

Theoretical Occupancy

Est. Speedup: 39.27%

The 4.00 theoretical warps per scheduler this kernel can issue according to its occupancy are below the hardware maximum of 12. This kernel's theoretical occupancy (33.3%) is limited by the number of required registers.

Key Performance Indicators

GPU and Memory Workload Distribution

Analysis of workload distribution in active cycles of SM, SMP, SMSP, L1 & L2 caches, and DRAM

Average SM Active Cycles [cycle]	1,372,031.54	Average L1 Active Cycles [cycle]	1,372,031.54
Average L2 Active Cycles [cycle]	119,217	Average SMSP Active Cycles [cycle]	1,373,077.21
Average DRAM Active Cycles [cycle]	76,952	Total SM Elapsed Cycles [cycle]	44,656,176
Total L1 Elapsed Cycles [cycle]	44,656,176	Total L2 Elapsed Cycles [cycle]	26,581,360
Total SMSP Elapsed Cycles [cycle]	178,624,704	Total DRAM Elapsed Cycles [cycle]	26,440,704

SMs Workload Imbalance

Est. Speedup: 19.31%

One or more SMs have a much higher number of active cycles than the average number of active cycles. Maximum instance value is 26.18% above the average, while the minimum instance value is 20.40% below the average.

Key Performance Indicators

SMSPs Workload Imbalance

Est. Speedup: 19.30%

One or more SMSPs have a much higher number of active cycles than the average number of active cycles. Maximum instance value is 26.15% above the average, while the minimum instance value is 20.95% below the average.

Key Performance Indicators

L1 Slices Workload Imbalance

Est. Speedup: 19.31%

One or more L1 Slices have a much higher number of active cycles than the average number of active cycles. Maximum instance value is 26.18% above the average, while the minimum instance value is 20.40% below the average.

Key Performance Indicators

Source Counters

Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.

Branch Instructions [inst]	420,964	Branch Efficiency [%]	95.13
Branch Instructions Ratio [%]	0.05	Avg. Divergent Branches [branches]	139.06

Uncoalesced Shared Accesses

Est. Speedup: 0.43%

This workload has uncoalesced shared accesses resulting in a total of 1295 excessive wavefronts (1% of the total 221075 wavefronts). Check the L1 Wavefronts Shared Excessive table for the primary source locations. The [🔗 CUDA Best Practices Guide](#) has an example on optimizing shared memory accesses.

Key Performance Indicators

L1 Wavefronts Shared Excessive

Location	Value	Value (%)
► 0x73ea872a76a0 in ik_batch_solve_mixed	1,295	100
► 0x73ea872aeb40 in ik_batch_solve_mixed	0	0
► 0x73ea872ad700 in ik_batch_solve_mixed	0	0
► 0x73ea872ad6e0 in ik_batch_solve_mixed	0	0
► 0x73ea872ad6c0 in ik_batch_solve_mixed	0	0

Follow the *rules outputs* to get guidance on how to navigate through the report and quickly discover performance bottlenecks in this kernel. You could also disable [individual sections](#) to focus on selected performance aspects and make profiling faster.